الملف الخامس: مدير الاتصالات (Connection Manager) والأمن والأتمتة

أولاً: مدير الاتصالات — الخادم الرئيسي للمنصة

مدير الاتصالات هو قلب النظام. يُوفّر خدمتين متزامنتين: خادم gRPC للتواصل مع الوكلاء، وخادم REST/HTTP للتواصل مع لوحة التحكم والخدمات الأخرى.

البنية الداخلية لمدير الاتصالات

يتبع مدير الاتصالات معمارية Clean Architecture مقسّمة إلى طبقات واضحة:

طبقة الواجهة الخارجية (Handlers/Transport): تحتوي على معالجات gRPC ومعالجات REST HTTP. هذه الطبقة تستقبل الطلبات الواردة، تُحوّلها من تنسيق الشبكة إلى هياكل البيانات الداخلية، وتستدعي طبقة الخدمات.

طبقة الخدمات (Services): تحتوي على منطق الأعمال الأساسي. كل خدمة تركز على مجال محدد:

- `AgentService` : إدارة دورة حياة الوكلاء (التسجيل، الحالة، المقاييس، العزل).
- `AuthService` : المصادقة وإدارة المستخدمين ورموز JWT.
- `CertificateService` : إصدار وإدارة الشهادات الرقمية.
- `AutomationService` : منطق قواعد الأتمتة وتنفيذ Playbooks.
- `CommandService` : إصدار الأوامر للوكلاء وتتبع حالتها.
- `NotificationService` : إرسال الإشعارات.
- `MetricsService` : تتبع مقاييس الأداء.
- `MLOptimizer` : تحسين أولويات قواعد الأتمتة بالتعلم الآلي.

طبقة المستودع (Repository): تُخفي تفاصيل قاعدة البيانات. كل نوع بيانات له repository interface يُعرِّف العمليات، وimplementation يُنفّذها عبر استعلامات PostgreSQL.

طبقة النماذج (Models): تعريفات هياكل البيانات (Agent, Alert, Command, User, etc).

ثانياً: آلية المصادقة والتفويض بالتفصيل

مصادقة المحللين (JWT)

عندما يُسجّل محلل دخوله من لوحة التحكم:

1. يُرسل اسم المستخدم وكلمة المرور إلى `POST /api/v1/auth/login` .
2. يجلب الخادم سجل المستخدم من PostgreSQL.
3. يتحقق أن الحساب غير مقفل (أقل من 5 محاولات فاشلة متتالية) وغير معطّل.
4. يُقارن كلمة المرور بالـ hash المُخزَّن باستخدام bcrypt . `bcrypt.CompareHashAndPassword()` هو خوارزمية hash مُصمَّمة لتكون بطيئة عمداً مما يجعل هجمات Brute Force غير عملية.
5. عند النجاح يُولِّد الخادم زوجاً من الرموز باستخدام مفتاح RSA خاص:
   - **Access Token:** صلاحيته 15 دقيقة، يُستخدَم في كل طلب API.
   - **Refresh Token:** صلاحيته 7 أيام، يُستخدَم فقط لتجديد Access Token.
6. يُسجَّل الحدث في سجل التدقيق (audit_logs).

كل طلب API يتضمن الـ Access Token في `Authorization: Bearer <token>` . الخادم يُحقق منه بثلاث خطوات:

- **أولاً:** هل التوقيع الرقمي صحيح؟ (باستخدام المفتاح العام RSA).
- **ثانياً:** هل الرمز في القائمة السوداء في Redis؟ (معناه المستخدم سجّل خروجه).
- **ثالثاً:** هل صلاحيته لم تنتهِ؟

مصادقة الوكلاء (mTLS + Agent ID)

الوكيل لا يستخدم كلمات مرور — بل يُقدّم شهادته الرقمية X.509 مع كل اتصال gRPC. الخادم يتحقق أن الشهادة:

- موقّعة من CA الخاص بالمنصة.
- لم تنتهِ صلاحيتها.
- لم تُلغَ (يُقارَن serial number مع قائمة الإلغاء).

بعد التحقق من الشهادة، يستخرج الخادم `agent_id` من metadata الطلب gRPC ويتحقق أن هذا الوكيل موجود ونشط في قاعدة البيانات.

ثالثاً: إدارة الشهادات الرقمية (PKI ذاتي التوقيع)

النظام يُشغّل بنية مفاتيح عامة (PKI) خاصة به:

CA الجذر: عند أول تشغيل للخادم، يتحقق من وجود `certs/ca.crt` و `certs/ca.key` . إذا لم يجدهما يُولِّد زوج مفاتيح RSA 4096-bit جديداً وشهادة CA ذاتية التوقيع صالحة لـ 10 سنوات. هذه الشهادة هي "جذر الثقة" للمنصة كلها.

إصدار شهادات الوكلاء: عندما يُقدّم وكيل جديد CSR (Certificate Signing Request):

1. يتحقق الخادم من صحة الـ CSR.
2. يُنشئ شهادة X.509 جديدة بـ serial number فريد.
3. يُوقِّعها بمفتاح CA الخاص.
4. يُضمِّن في الشهادة: agent_id كـ hostname، SubjectAltName، تاريخ الإصدار والانتهاء (سنة واحدة).
5. يُخزِّن سجل الشهادة في جدول `certificates` .
6. يُعيد الشهادة الموقّعة + شهادة CA للوكيل.

تجديد الشهادات: كل نبضة قلب (Heartbeat) يتحقق الخادم من تاريخ انتهاء شهادة الوكيل. إذا كانت ستنتهي خلال 30 يوماً، يُبلِّغ الوكيل في استجابة الـ Heartbeat. الوكيل يُنشئ CSR جديد ويُرسله للخادم للحصول على شهادة جديدة.

رابعاً: منظومة الأتمتة (Automation System)

نظام الأتمتة يُتيح الاستجابة التلقائية للتنبيهات دون تدخل بشري. يتكوّن من طبقتين:

الطبقة الأولى: Playbooks (كتب اللعب)

Playbook هو سكريبت استجابة مُعرَّف مسبقاً يتضمن تسلسلاً من الأوامر. مثلاً "Ransomware Response Playbook" قد يحتوي على:

1. عزل الجهاز عن الشبكة.
2. إيقاف العمليات المشبوهة.
3. جمع السجلات للتحقيق.
4. إرسال تنبيه للمحلل.

كل أمر له `timeout` و (stop/continue) `on_failure`. الـ Playbook يُخزَّن في قاعدة البيانات كـ JSON ويمكن إنشاؤه وتعديله من لوحة التحكم.

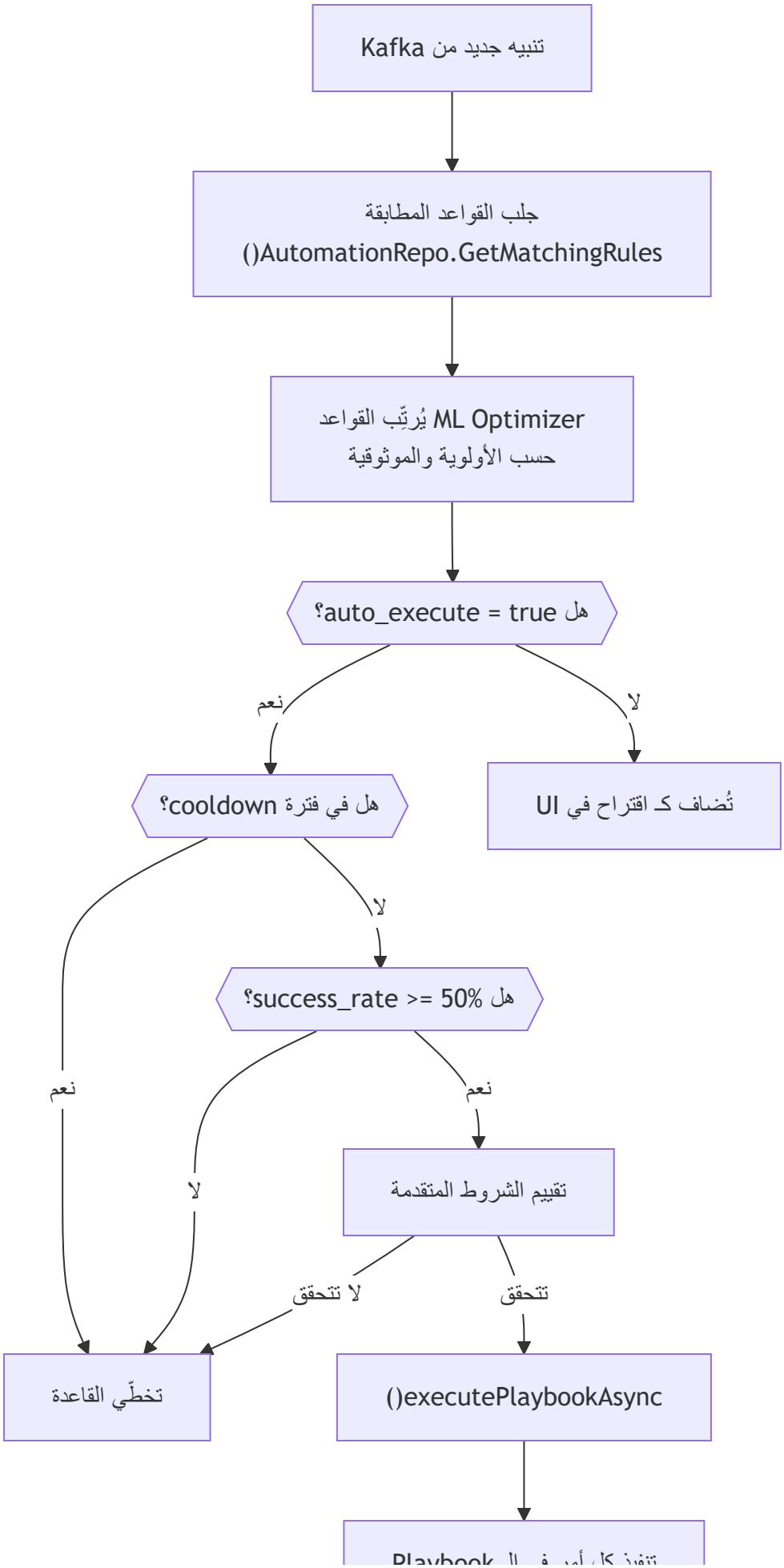
الطبقة الثانية: Automation Rules (قواعد الأتمتة)

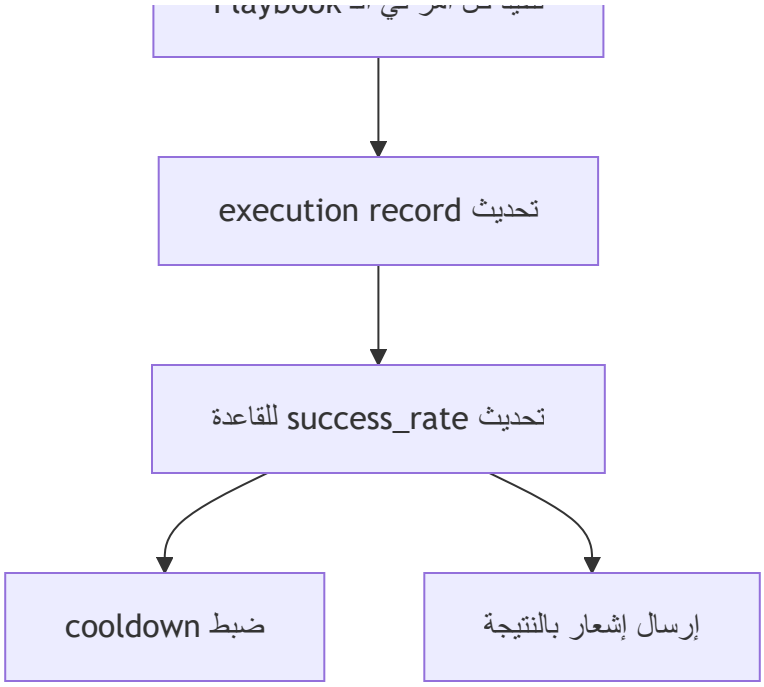
القاعدة تُعرِّف: **متى** يُنفَّذ Playbook. تحتوي على:

- **trigger_conditions:** شروط JSON تحدد متى تُطابق تنبيهاً. مثلاً:
  `{severity": ["critical", "high"], "rule_patterns": ["ransomware", "malware"]"}` .
- **playbook_id:** الـ Playbook المُنفَّذ عند التطابق.
- **auto_execute:** هل تُنفَّذ تلقائياً أم تحتاج موافقة بشرية؟
- **cooldown_minutes:** الحد الأدنى للوقت بين تنفيذين متتاليين للقاعدة (لتجنب التشغيل المتكرر).

- **priority:** ترتيب التطبيق عند تطابق قواعد متعددة.
- **success_rate:** معدل نجاح القاعدة التاريخي (يتحكم في ما إذا كان يجب تخطّيها).

كيف تعمل الأتمتة عند وصول تنبيه؟


تنبيه جديد من Kafka
جلب القواعد المطابقة
()AutomationRepo.GetMatchingRules
ML Optimizer يُرتِّب القواعد
حسب الأولوية والموثوقية
هل auto_execute = true؟
نعم
لا
هل في فترة cooldown؟
تُضاف كـ اقتراح في UI
لا
هل success_rate >= 50%؟
نعم
نعم
لا
تقييم الشروط المتقدمة
لا تتحقق
تتحقق
تخطّي القاعدة
()executePlaybookAsync



ML Optimizer — الذكاء الاصطناعي في الأتمتة

`MLOptimizer` هو مكوّن يُعيد ترتيب أولويات قواعد الأتمتة ديناميكياً بناءً على:

- معدل النجاح التاريخي لكل قاعدة.
- خصائص التنبيه الحالي (خطورة، اسم القاعدة، نقاط المخاطرة).
- السياق المحيط (هل الجهاز في الإنتاج؟ هل هناك هجمات مماثلة على أجهزة أخرى؟).

كما يُقترح `MLOptimizer` تحسينات للقواعد الموجودة: هل يجب رفع الـ priority؟ هل يجب زيادة الـ cooldown؟ هل قاعدة ذات معدل نجاح منخفض يجب تعطيلها؟

خامساً: نظام اقتراح Playbooks الذكي

عند استلام تنبيه، يُولِّد `AutomationService` قائمة من الـ Playbooks المُقترَحة حتى لو لم تكن هناك قاعدة تطابق تلقائياً. الاقتراحات تعتمد على:

- خطورة التنبيه: التنبيهات Critical تقترح "Immediate Containment Playbook" بثقة 95%.
- محتوى اسم القاعدة: إذا احتوى على "malware" أو "trojan" يُقترح "Malware Analysis Playbook".
- نقاط المخاطرة: إذا تجاوزت 8.0 يُقترح "Comprehensive System Scan".
- كلمات محددة مثل "ransomware" أو "encryption" تُقترح "Ransomware Attack Response Playbook" بثقة 98%.

سادساً: إدارة الثغرات الأمنية (Vulnerability Management)

مدير الاتصالات يدمج بيانات الثغرات الأمنية من مصدرين:

أولاً — الثغرات من الوكيل: الوكيل يُشغّل `vuln_scanner.go` الذي يفحص البرامج المثبّتة على الجهاز (من Registry) ويُقارنها بقائمة CVEs المعروفة. النتائج تُرسَل كأحداث للخادم.

ثانياً — KEV Sync (Known Exploited Vulnerabilities): الخادم يُزامن دورياً قائمة `kev_sync_service.go` من CISA (وكالة الأمن السيبراني الأمريكية) التي تحتوي على الثغرات المستغلة فعلياً في الطبيعة.

حساب درجة الأولوية: كل ثغرة مكتشفة تحصل على `priority_score` يجمع بين:

- درجة CVSS (شدة الثغرة).
- حساسية الجهاز (criticality).
- بيئة الجهاز (production/staging/development).
- هل الثغرة في قائمة KEV (هذا يرفع الأولوية بشكل كبير).

هذا يمنح المدراء قائمة مرتبة بأكثر الثغرات إلحاحاً للمعالجة، مع التركيز على الثغرات التي تجمع بين أجهزة حساسة وثغرات مستغلة فعلياً.

سابعاً: سجلات التدقيق (Audit Logs)

كل فعل ذي أهمية في النظام يُسجَّل في جدول `audit_logs` . السجل يتضمن:

- من قام بالفعل (user_id, username).
- ما هو الفعل (login_success, login_failed, agent_registered, agent_suspended, command_issued, playbook_executed, etc).
- على أي كيان (entity_type: agent/user/playbook، entity_id).
- متى (timestamp).
- هل نجح (success: true/false).
- تفاصيل إضافية (details: "hostname: SERVER-01").

هذا يُوفّر أثراً كاملاً لكل ما حدث في النظام، مما يدعم متطلبات الامتثال (GDPR, ISO 27001, SOC 2) ويساعد في التحقيق عند وقوع حادث.

ثامناً: Load Tests وتقييم الأداء

في مجلد `/connection-manager/load_tests` توجد اختبارات حمل مكتوبة بـ Go تُحاكي عدداً كبيراً من الوكلاء المتصلة في آن واحد. الاختبارات تقيس: زمن استجابة gRPC، throughput (عدد الأحداث في الثانية)، استخدام الذاكرة والـ CPU تحت الحمل.

تاسعاً: الأسئلة التقنية المتوقعة من اللجنة

كيف يمنع النظام انتحال هوية الوكيل؟

يمنعه mTLS. كل وكيل يحمل شهادة رقمية X.509 موقّعة حصراً من CA الخاص بالمنصة. المهاجم لا يستطيع تزوير هذه الشهادة لأنه لا يملك المفتاح الخاص للـ CA (المحفوظ على الخادم فقط). حتى لو اعترض المهاجم حزمة بيانات مُرسَلة من وكيل شرعي، لا يستطيع إعادة تشغيلها لأن TLS يُضمّن رموزاً عشوائية تمنع هجمات Replay.

ماذا لو اخترق المهاجم الوكيل وحاول تعطيله؟

الوكيل يحمي نفسه بثلاث طبقات: ACLs على مجلد البيانات، ACLs على مفتاح Registry، وعمل بصلاحيات SYSTEM. لكن إذا نجح المهاجم في إيقاف خدمة الوكيل، سيلاحظ الخادم انقطاع الـ Heartbeat خلال 60 ثانية ويُغيّر حالة الجهاز لـ "offline" ويُطلق تنبيهاً للمحلل. هذا الغياب نفسه يُعدّ مؤشراً أمنياً.

هل يمكن للمهاجم إرسال أحداث مزيّفة للخادم؟

لا. كل حدث يأتي عبر اتصال gRPC مُصادَق عليه بمفتاح mTLS خاص بالوكيل. بدون هذه الشهادة الرقمية الصحيحة، لن يقبل الخادم أي اتصال.